

URL Shortener

A backend service for shortening long URLs and redirecting users through generated short codes.

The project is implemented in Go using Clean Architecture principles, with a focus on separation of concerns, testability, and maintainable design.

Features

- Shorten long URLs into unique short codes
- Redirect users from short codes to original URLs
- Retrieve metadata for shortened links
- Return existing short codes for previously shortened URLs
- Validate and normalize URLs
- Support multiple persistence implementations:
 - In-memory storage
 - PostgreSQL storage
- RESTful HTTP API
- Unit and integration tests
- Race detection testing
- Benchmark and profiling support
- Thread-safe repository operations

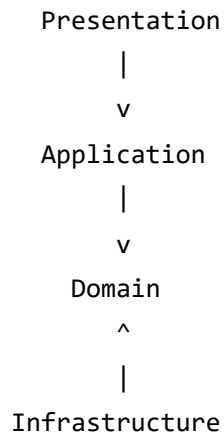
Tech Stack

- **Language:** Go 1.22+
- **HTTP Framework:** Gin
- **Database:** PostgreSQL
- **ORM:** GORM
- **Architecture:** Clean Architecture
- **Containerization:** Docker Compose
- **Testing:** Go testing package, race detector, benchmarks

Architecture

The project follows Clean Architecture principles to keep business logic independent from external frameworks, databases, and delivery mechanisms.

The dependency direction is:



Outer layers depend on inner layers through abstractions defined by the core layers.

Layers

Domain

Contains the core business entities and interfaces.

Responsibilities:

- Define business models
- Define repository contracts
- Contain domain-level errors

The domain layer has no dependency on external frameworks or infrastructure.

Application

Contains the business use cases.

Responsibilities:

- URL shortening logic
- URL validation and normalization
- Short code generation
- Duplicate URL handling
- Link retrieval

This layer depends only on domain abstractions.

Infrastructure

Contains implementations of external systems.

Responsibilities:

- Database connection
- PostgreSQL repositories
- In-memory repositories
- Data persistence models

Infrastructure implements the interfaces defined in the domain layer.

Presentation

Handles communication with clients.

Responsibilities:

- HTTP routing
- Request handling
- Response formatting
- HTTP error handling

Project Structure

```
.
├── cmd
│   └── server
│       └── main.go
├── bootstrap
│   └── app.go
├── internal
│   ├── application
│   │   ├── dto
│   │   └── usecase
│   ├── domain
│   │   ├── entities
│   │   ├── errors
│   │   └── ports
│   ├── infrastructure
│   │   ├── database
│   │   └── persistence
│   │       ├── memory
│   │       └── postgres
│   └── presentation
│       ├── controller
│       ├── middleware
│       └── routes
├── docker-compose.yml
├── DECISIONS.md
└── README.md
```

Getting Started

Prerequisites

- Go 1.22+
- Docker
- Docker Compose

Run with Docker

Start PostgreSQL:

```
docker compose up -d
```

Run the application:

```
go run ./cmd/server
```

The server will start on:

```
http://localhost:8080
```

API Endpoints

Shorten URL

POST /api/shorten

Creates a shortened URL.

Request:

```
{  
  "url": "https://example.com"  
}
```

Response:

```
{  
  "code": "aB12cd",  
  "short_url": "http://localhost:8080/aB12cd"  
}
```

Possible responses:

Status Code	Description
201	URL shortened successfully
400	Invalid URL
500	Internal server error

Redirect

GET /{code}

Redirects the user to the original URL.

Example:

```
GET /aB12cd
```

Possible responses:

Status Code	Description
302	Redirect successful
404	Short code not found

Get Link Metadata

GET /api/links/{code}

Returns information about a shortened URL.

Example:

```
GET /api/links/aB12cd
```

Response:

```
{
  "code": "aB12cd",
  "url": "https://example.com",
  "created_at": "2026-10-09T12:00:00Z"
}
```

Possible responses:

Status Code	Description
200	Link metadata returned successfully
404	Short code not found
500	Internal server error

Repository Pattern

The application depends on repository interfaces defined in the domain layer.

Available implementations:

Memory Repository

Used for:

- Unit testing
- Development
- Local execution without a database

PostgreSQL Repository

Used for:

- Persistent storage
- Production environments

The application layer does not know which repository implementation is being used.

Short Code Generation

Short codes are generated using a random alphanumeric generator.

Generated codes:

- Contain uppercase letters, lowercase letters, and numbers
- Are checked for uniqueness before saving
- Are regenerated if a collision occurs

Error Handling

The project uses domain-level errors to separate business logic from HTTP concerns.

Examples:

- Invalid URL
- Link not found
- Duplicate URL
- Repository failures

Errors are converted into appropriate HTTP responses in the presentation layer.

Concurrency and Race Safety

The project is designed to support concurrent requests safely.

Race detection is used to verify repository and application behavior:

```
go test -race ./...
```

Repository implementations use synchronization mechanisms to protect shared state.

Testing

Run all tests:

```
go test ./...
```

Run tests with race detection:

```
go test -race ./...
```

Generate coverage report:

```
go test -coverprofile=coverage.out ./...
```

Open coverage report:

```
go tool cover -html=coverage.out
```

Benchmarks

Benchmarks were measured on:

CPU: Intel Core i7-13620H

Example benchmark results:

Operation	Result
URL Shortening	~1.9 μ s/op
Redirect Lookup	~41 ns/op

Run benchmarks:

```
go test ./internal/application/usecase -bench=. -benchmem
```

Usage Example

Create a shortened URL:

```
curl -X POST http://localhost:8080/api/shorten \  
-H "Content-Type: application/json" \  
-d '{"url":"https://example.com"}'
```

Example response:

```
{  
  "code": "aB12cd",  
  "short_url": "http://localhost:8080/aB12cd"  
}
```

Opening the generated URL redirects to the original address.

Design Decisions

Important architectural and implementation decisions are documented in:

[DECISIONS.md](#)